

Relatório Técnico: Extração de Características de Imagens Utilizando OpenCV e Python

Desenvolvido em Python com OpenCV e NumPy

1 Introdução

Este relatório descreve um código desenvolvido em Python que utiliza a biblioteca OpenCV para extrair características de imagens, aplicando técnicas como Cadeia de Código (Code Chain) e Padrões Binários Locais (Local Binary Pattern - LBP). O objetivo é analisar imagens e armazenar suas características em vetores, que podem ser usados em processos de classificação ou reconhecimento.

2 Objetivo

O código tem como finalidade processar imagens e extrair suas principais características utilizando técnicas de processamento de imagens. Ele realiza as seguintes operações:

- Aplica **Cadeia de Código** (`HistCodeChain`) para capturar padrões estruturais das bordas da imagem.
- Utiliza o **Padrão Binário Local** (`calcula_pixel_lbp`) para análise de textura e características locais da imagem.
- Gera histogramas normalizados dessas características para cada imagem processada.
- Lê um conjunto de imagens, processa-as e salva os dados em listas para posterior análise.

3 Dependências e Bibliotecas Utilizadas

O código depende das seguintes bibliotecas:

- **OpenCV** (`cv2`): Para carregamento, manipulação e processamento de imagens.
- **NumPy** (`numpy`): Para cálculos matemáticos e manipulação de arrays.
- **CSV**: Para leitura dos rótulos das imagens a partir de um arquivo externo.
- **OS**: Para manipulação de diretórios e arquivos do sistema.
- **argparse**: Para permitir a passagem de argumentos via linha de comando.

4 Funcionamento do Código

O código é estruturado da seguinte forma:

4.1 Leitura de Argumentos

Os argumentos são lidos via `argparse`:

```
parser = ap.ArgumentParser()
parser.add_argument("-t", "--trainingSet", help="Path_to_Training_Set", required=True)
parser.add_argument("-l", "--imageLabels", help="Path_to_Image_Label_Files",
                    required=True)
args = vars(parser.parse_args())
```

Os parâmetros esperados são:

- `-t (--trainingSet)`: Diretório contendo as imagens de treino.
- `-l (--imageLabels)`: Arquivo CSV com os rótulos das imagens.

4.2 Cadeia de Código (HistCodeChain)

Esta função aplica um limiar (`threshold`) para converter a imagem em binária e, em seguida, detecta os contornos. Após isso, calcula a Cadeia de Código para capturar a direção do movimento entre pixels vizinhos nos contornos detectados, retornando um histograma normalizado.

```
def HistCodeChain(img):
    ret, thresh = cv2.threshold(img, 127, 255, 0)
    thresh = abs(thresh - 255)
    contours, hierarchy = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                                          cv2.CHAIN_APPROX_NONE)
```

O código percorre os contornos e armazena as direções das mudanças de pixel em um histograma.

4.3 Padrão Binário Local (LBP)

A função **LBP (Local Binary Pattern)** calcula padrões de textura para cada pixel da imagem, comparando seu valor com os dos pixels vizinhos:

```
def calcula_pixel_lbp(img, x, y):
    center = img[x][y]
    val_ar = []
    val_ar.append(pega_pixel(img, center, x-1, y+1)) # acima direita
    val_ar.append(pega_pixel(img, center, x, y+1)) # direita
    val_ar.append(pega_pixel(img, center, x+1, y+1)) # abaixo direita
    val_ar.append(pega_pixel(img, center, x+1, y)) # abaixo
    val_ar.append(pega_pixel(img, center, x+1, y-1)) # abaixo esquerda
    val_ar.append(pega_pixel(img, center, x, y-1)) # esquerda
    val_ar.append(pega_pixel(img, center, x-1, y-1)) # acima esquerda
    val_ar.append(pega_pixel(img, center, x-1, y)) # acima
```

Cada pixel recebe um código LBP baseado na comparação com seus vizinhos, gerando uma nova imagem que representa a textura da original.

4.4 Processamento das Imagens

O código percorre todas as imagens no diretório especificado, converte-as para escala de cinza e aplica os métodos **LBP** e **HistCodeChain**:

```
for train_image in train_images:
    im = cv2.imread(train_image)
    img = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)

    hist = calc_prob_density(lbp)
    histchan = HistCodeChain(img)

    histograma = hist.tolist() + histchan.tolist()
    X_name.append(train_image)
    X_test.append(histograma)
    y_test.append(train_dic[os.path.split(train_image)[1]])
```

O resultado final é uma lista com os vetores de características extraídos de cada imagem.

5 Conclusão

Este código implementa um processo robusto para extração de características de imagens usando OpenCV e NumPy. As principais técnicas utilizadas foram:

- **HistCodeChain**: Extração de padrões estruturais baseados em contornos.
- **Local Binary Pattern (LBP)**: Extração de características de textura.
- **Geração de histogramas normalizados**: Representação eficiente dos dados.

Essas técnicas podem ser aplicadas em reconhecimento de padrões, classificação de imagens e outros domínios do processamento de imagens.